Feasibility Report: RAG-Based Self-Hosted Chatbot Deployment

Title: Feasibility Report: RAG-Based Self-Hosted Chatbot Deployment

Prepared by: Devdas Karambalkar

Reviewed by:, Suraj Nagraj

Version: v1.0

Table of Contents

Section		Title	Page No.
1		Introduction	3
	1.1	Problem Statement	3
	1.2	Objectives	3
2		Key Technologies and Current Setup	4
	2.1	Overview of Key Technologies and Components	4
3		Infrastructure & Software Requirement Specification	5
	3.1	Hardware Requirements (Current and Recommended)	5
	3.2	Software Requirements (Current and Recommended)	7
4		Architecture & Methodology	8
	4.1	System Architecture (Block Diagram for RAG Chatbot)	8
	4.2	Methodology (Isolated LAN and Multi-System Testing)	9
7		Results & Analysis	14
	5.1	LLM Inference Latency Deep Dive	14
	5.2	Containerization and Network Overhead	14
	5.3	Chroma DB Performance Considerations	15
	5.4	RAG System Efficiency	15
8		Risks and Limitations	16
	8.1	Technical Limitations	16
	8.2	Operational Constraints	16
	8.3	Mitigation Strategies	16
9		Conclusion	18

1. Introduction

1.1 Problem Statement

The business unit requires a self-hosted Retrieval-Augmented Generation (RAG) based chatbot capable of operating within an on-premise environment. They also need the ability to upload their own documents and receive answers based on that content.

1.2 Objectives

- To evaluate the current prototype's architecture and performance, specifically analysing the observed response latency.
- To identify the root causes of the performance bottlenecks.
- To provide actionable recommendations for optimizing performance and ensuring the system's readiness for production deployment in an on-premise environment.

2. Key Technologies and Current Setup

2.1 Overview of Key Technologies and Components

The system constitutes of all the components shown in Fig. 4.1: RAG Chatbot Architecture Block Diagram and it utilizes a multi-container Docker setup, designed to provide a robust and portable application stack. Key components include:

- Self-hosted-chatbot-backend: This container, based on the code-chatbot image, exposes a Python API on port 8002. It serves as the central application logic, orchestrating interactions between the user interface, the LLM, and the vector database.
- Self-hosted-chatbot-frontend: Running on the code-frontend image, this container provides the web-based user interface, accessible via port 8003.
- ollama-llm: This container utilizes the Ollama/Ollama:latest image and hosts the DeepSeek R1 7B LLM [Limited Support on Nordic languages], handling all inference requests on port 11434 locally. Used it for only to test the feasibility study on the use case.
- Chroma-vectordb: Based on chromadb/chroma:latest, this container functions as the vector store for the RAG component, accessible on port 8001 locally.

The core LLM in use is the DeepSeek R1 7B model. Its specifications, as detailed in the "Model Comparison Analysis", provide critical insights into its resource demands:

DeepSeek R1 7B Model Specifications and Resource Demands

Specification	Details
Model Name	deepseek-r1:7b-qwen-distill-q4_K_M
Architecture	Qwen2
Parameters	7.6 billion
Model Size	4.7 GB
Context Length	131,072 tokens
Quantization	Q4_K_M (4-bit mixed precision)
Memory Required at Runtime	~6-8 GB
Current Memory Usage (Ollama container)	8.83 GiB (57% of Docker allocation)

Table 2.1: Summary of DeepSeek R1 7B Model

3. Infrastructure & Software Requirement Specification

3.1 Hardware Requirements (Current and Recommended)

The project is hosted on a system with the following key specifications:

Hardware setup for Feasibility study on "**Locally hosted Ai Assistant**":

Component	Current Specification	Impact on LLM Inference
Operating System	Microsoft Windows 11 Enterprise	Higher overhead compared to Linux for server-grade AI workloads, especially with Docker Desktop's virtualization layer.
Processor	11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz, 4 Cores, 8 Logical Processors	Mobile-grade CPU, limited parallel processing capabilities and lower memory bandwidth for demanding LLM workloads.
Installed Physical Memory (RAM)	32.0 GB	Good quantity, but *available* RAM is critically insufficient for the LLM.
Available Physical Memory (RAM)	7.82 GB	Primary cause of severe memory thrashing and extreme latency.
Total Virtual Memory	40.8 GB	Indicates reliance on disk-based virtual memory to compensate for insufficient physical RAM.
Page File Space	9.08 GB	Confirms active use of slow disk I/O for memory management.

Table 3.1: Summary of Host System Specifications and LLM Compatibility

Recommended Hardware Upgrades for Team Blue Deployment:

Component	Current Specification	Recommended Specification	Justification / Impact
Processor (CPU)	11th Gen Intel Core i7-1165G7 (4-core)	**Dual-socket AMD EPYC (e.g., 9334)** or **Intel Xeon / Ryzen 9 / Core i9** (16+ cores)	Facilitates parallel data handling, multi-GPU coordination, and model loading. Dual-EPYC delivers ~5–8 tokens/sec performance on CPU-only setups (huggingface.co).
Installed Physical Memory (RAM)	32 GB (7.8 GB available)	**128 – 192 GB** (for GPU setups), or **384 GB+** for CPU-only installations	Avoids swapping and supports high throughput. 180 GB unified memory enables 5+ tokens/sec on single GPU . 384 GB supports CPU-only at 5–8 tkn/s .
Graphics Processing Unit (GPU)	Integrated Iris Xe (unused)	**NVIDIA RTX 4090 / A6000 (24 GB VRAM)** – standard; **A100 / H100 (40 GB+ VRAM)** – enterprise grade	24 GB VRAM GPUs support quantized 30B models, delivering ~3–5 tokens/sec. A100/H100 boost throughput and multi-user capacity.
Storage Type	Unspecified HDD/SSD	**>200 GB NVMe SSD** (model + Docker/OSS volumes)	Ensures fast model load, offloading, and prevents I/O bottlenecks.

Table 3.2: Recommended Hardware for LLM-based RAG Deployment

Note: The recommendations provided are based on currently available online data and identified needs. However, they may require modification or reconfiguration based on the final solution approach, the availability of Team Blue resources, and evolving stakeholder requirements.

3.2 Software Requirements (Current and Recommended)

Current Software Stack:

- **Programming Language:** Python (implied by Backend container).
- **Container Images:** code-chatbot, code-frontend, ollama/ollama:latest, chromadb/chroma:latest.
- **Operating System:** Microsoft Windows 11 Enterprise.

Software and Configuration Tuning Recommendations:

- **Ollama and Model-Specific Optimizations:** Reduce Context Window (e.g., to 4096 or 8192 tokens) , Leverage GPU (if added), Keep Ollama Updated.
- **Docker Networking and Resource Allocation Adjustments:** Consider Host Network Mode (--net=host) for ollama-llm and self-hosted-chatbot-backend in isolated environments , Maximize Docker Desktop allocated CPU cores and RAM.
- **ChromaDB Performance Tuning:** Monitor its memory footprint to ensure the embedding index remains in RAM as the knowledge base grows.

4. Architecture & Methodology

4.1 System Architecture (Block Diagram for RAG Chatbot)

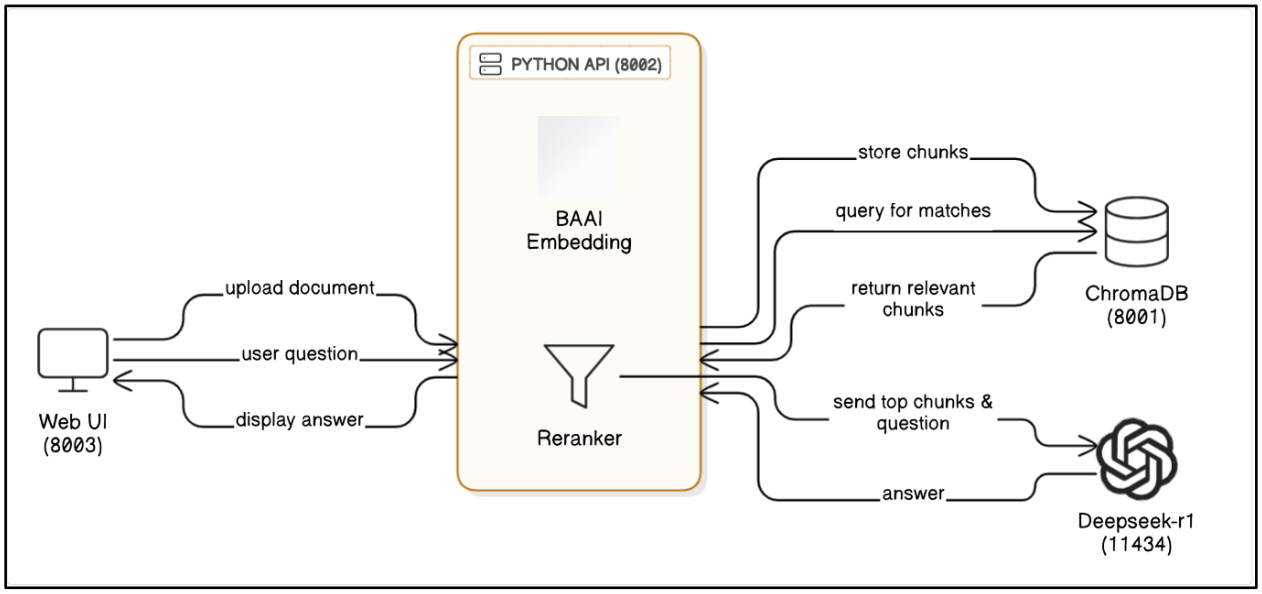

Fig. 4.1: RAG Chatbot Architecture Block Diagram

The system architecture for the RAG chatbot involves a multi-container Docker setup. The Frontend (Web UI, Port: 8003) communicates with the Backend (Python API, Port: 8002). The Backend, in turn, interacts with Ollama (AI Model, Port: 11434) and ChromaDB (Vector DB, Port: 8001).

Description of Architecture Components:

- **Frontend (self-hosted-chatbot-frontend):** Provides the web-based user interface, accessible via port 8003.
- **Backend (self-hosted-chatbot-backend):** Serves as the central application logic, orchestrating interactions between the user interface, the LLM, and the vector database.
- **Ollama (ollama-llm):** Hosts the DeepSeek R1 7B LLM, handling all inference requests.
- **ChromaDB (chroma-vectordb):** Functions as the vector store for the RAG component.
- The adoption of Dockerization offers notable benefits, including environmental isolation, simplified dependency management, and enhanced portability, all of which are advantageous for on-premise deployment.

- **BAAI**: We are using BAAI models for both embedding generation and reranking tasks. These models provide high-quality semantic representations and help improve the

4.2 Methodology (Isolated LAN and Multi-System Testing)

To simulate an on-premise deployment and test the chatbot's functionality in an isolated environment without internet access, a dedicated Local Area Network (LAN) was established using a TENDA Wi-Fi router. This setup involved three distinct systems:

1. **Server System:** This system, which is the primary host detailed in this report (running Windows 11 Enterprise with an Intel Core i7-1165G7 processor and 32GB RAM), was configured to run all the Dockerized components: the self-hosted-chatbot-backend, self-hosted-chatbot-frontend, ollama-llm, and chroma-vectordb containers. It acted as the central hub for the entire application.

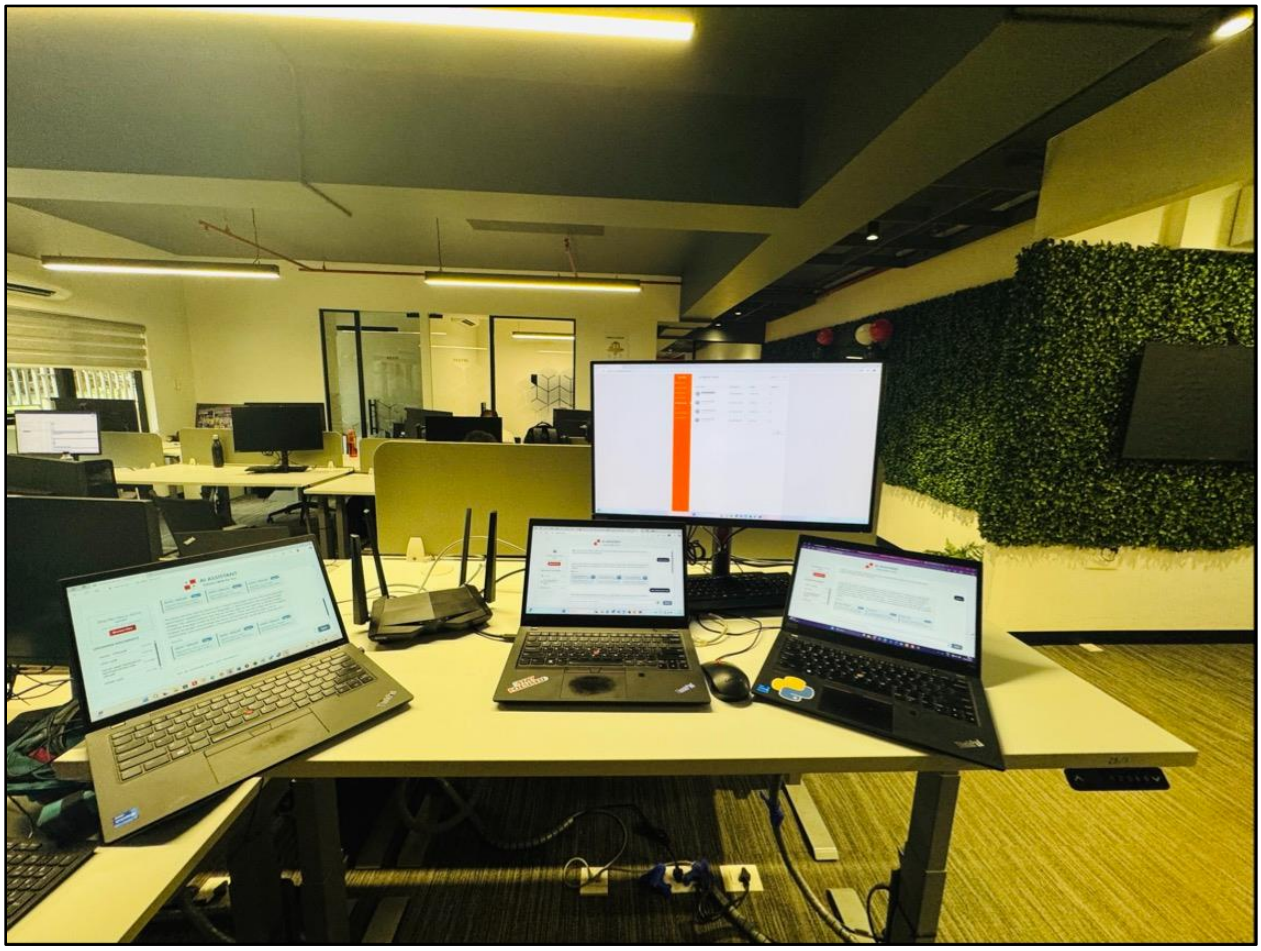

Image 4.2: Local Wi-Fi Setup using TENDA Router (Overview)

Note: Testing was conducted over a LAN environment with no machines connected to the internet.

2. **Client Systems (Two):** Two additional systems were connected to the TENDA Wi-Fi network. These systems served as client machines to access the chatbot application hosted on the server system.

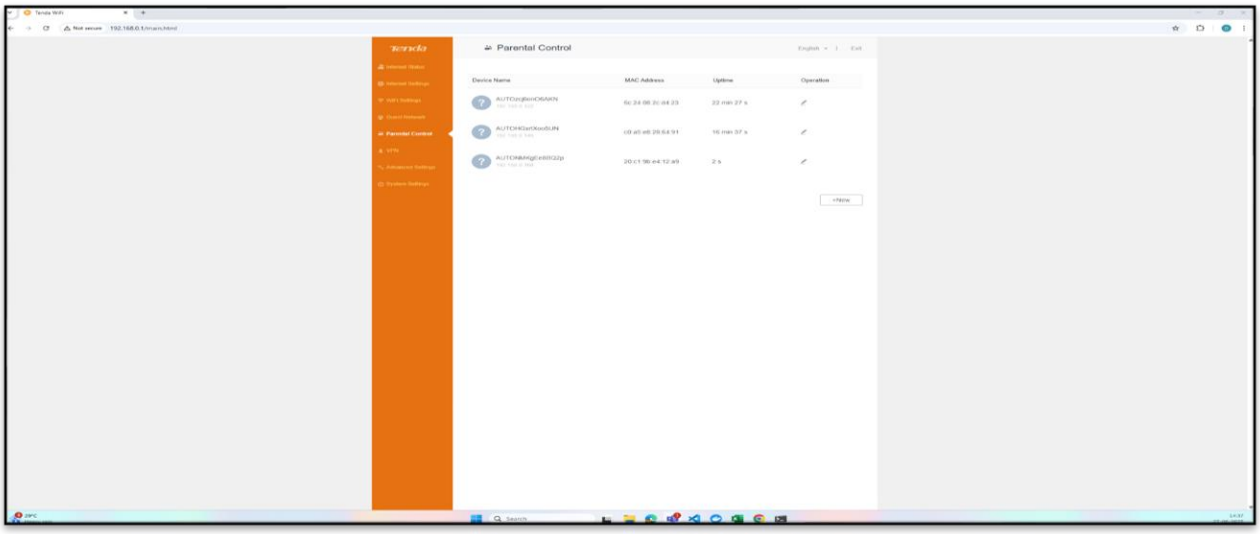

Image. 4.3: TENDA Wi-Fi Setup with One Server and Two Clients

3. **Testing Objective:** The testing was specifically conducted to verify the application's operation in an environment completely isolated from the public internet, fulfilling the business unit's on-premise requirement.

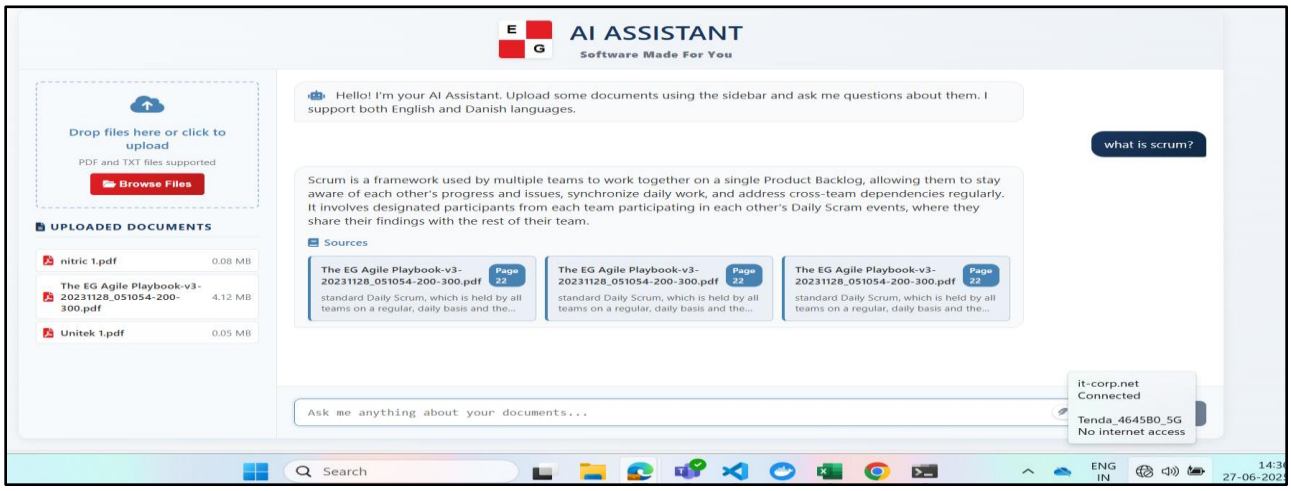

Image.4.4: Chatbot Response Example – Test Scenario 1

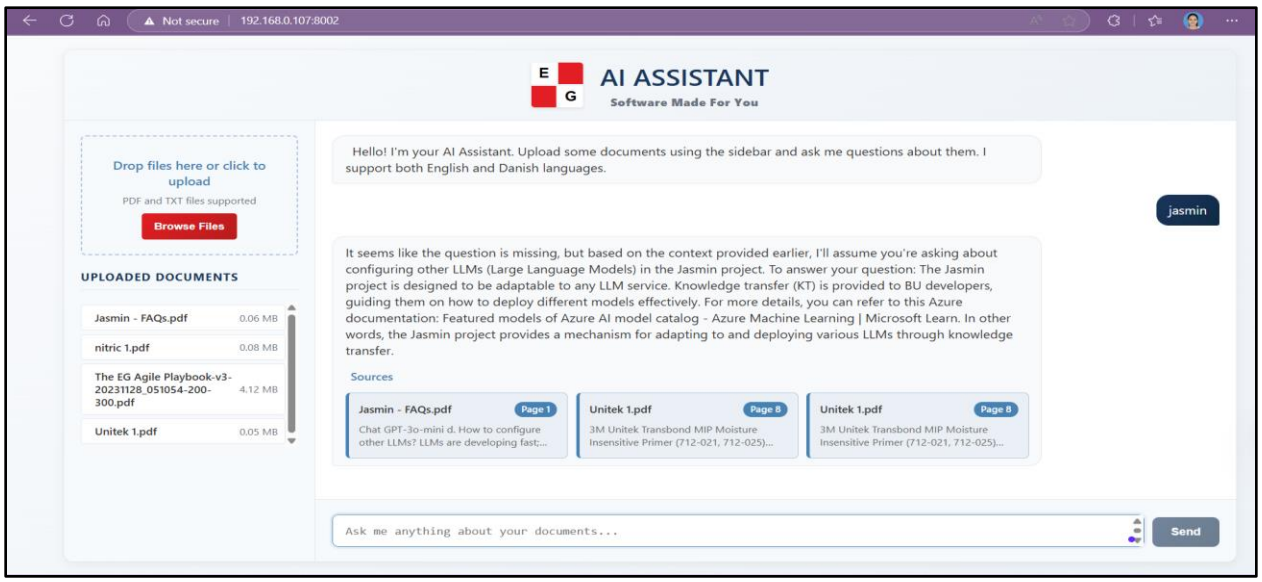

Image.4.5: Chatbot Response Example – Test Scenario 2

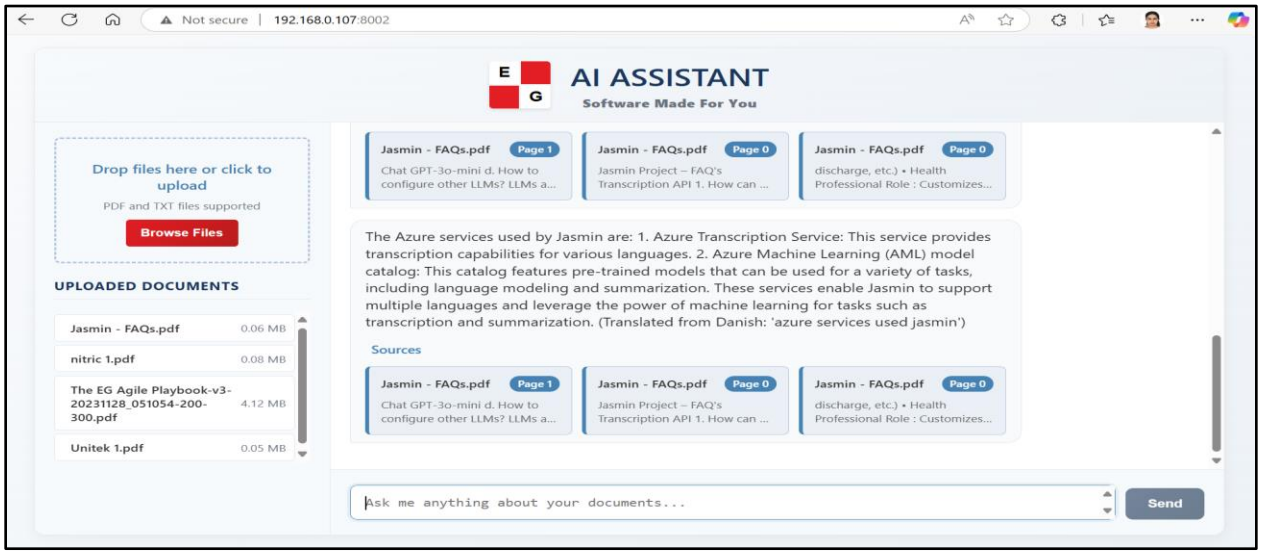

Image. 4.6: Chatbot Response Example – Test Scenario 3

5.Observed Results

- **Excessive Latency:**
  During multi-system LAN testing, average response times consistently exceeded **60 seconds**, even without internet dependency—highlighting severe system-level inefficiencies.
- **Performance Bottleneck Identified:**
  The delay is primarily due to the **server's inability to efficiently process LLM inference**. Specifically:
  - **Memory thrashing** caused by only **7.8 GB of available RAM**, while the Ollama container requires ~8.8 GB.
  - The **mobile-grade Intel i7-1165G7 CPU** is not optimized for parallel, high-throughput AI workloads.
  - The **DeepSeek R1 7B model's long 131,072-token context window** intensifies memory and compute pressure.
- **Environment-Independent Issue:**
  The system operated in a **completely isolated LAN**, confirming that network or internet factors are **not contributing** to the latency.
- **Containerization Overhead Minimal:**
  Docker networking introduced only **minor, millisecond-scale delays**, negligible compared to the server's inference lag.
- **ChromaDB Not a Bottleneck (Currently):**
  ChromaDB's memory usage is minimal (~7 MB), but performance may degrade if embedding volume scales without hardware improvements.

6.Future Considerations

- **Hardware Upgrades:**
  - **RAM:** Upgrade to at least **128–192 GB** (or 384 GB for CPU-only setups) to eliminate memory swapping.
  - **GPU:** Integrate a **dedicated GPU** (e.g., NVIDIA RTX 4090 or A100) with 24 GB+ VRAM to accelerate inference and support larger models.
  - **CPU:** Replace mobile-grade CPU with a **server-grade multi-core processor** (e.g., AMD EPYC or Intel Xeon).
  - **Storage:** Use **high-speed NVMe SSD** for faster model load and disk I/O.
- **Software & Configuration Tuning:**
  - Reduce LLM **context window** to ~4096–8192 tokens.
  - Use --net=host Docker setting in isolated environments to minimize containerization overhead.
  - Monitor and optimize **ChromaDB memory usage** as the dataset grows.
- **RAG Pipeline Optimization:**
  - Implement **smart chunking, re-ranking**, and **prompt engineering**.
  - Create a **systematic evaluation framework** using golden test cases to measure **accuracy**, **language compatibility**, and **response quality**.
- **Production Readiness Planning:**
  - Transition to **enterprise-grade infrastructure** with high availability.
  - Integrate **robust monitoring, centralized logging**, and **proactive alerting**.
  - Establish strict **security controls** and **regular maintenance** procedures.

7. Results & Analysis

7.1 LLM Inference Latency Deep Dive

The overwhelming evidence indicates that the Large Language Model (LLM) inference process within the ollama-llm container is the primary bottleneck responsible for the "more than one minute" response time.

The most significant factor contributing to this extreme delay is the insufficient available RAM and the resulting memory thrashing. The Ollama container's 8.8 GB memory requirement critically exceeds the 7.82 GB of available physical RAM on the host system [Image 3, Image 5]. This forces the operating system into continuous memory swapping to the disk's page file. LLM inference is a highly memory-intensive and bandwidth-hungry operation. When these operations are forced to rely on disk, which is orders of magnitude slower than RAM, the system enters a state of severe "thrashing." This condition fundamentally cripples performance, directly accounting for the extreme latency observed.

Even if the memory thrashing were entirely resolved, the Intel i7-1165G7 CPU, a mobile-grade processor, is not architecturally optimized for the highly parallel computations and high memory bandwidth required by a 7.6 billion parameter LLM.While 4-bit quantization helps reduce the model's footprint, it cannot overcome the fundamental speed limitations of CPU inference for models of this scale.

Furthermore, the 131,072 token context length of the DeepSeek model [Image 1] exacerbates the memory issue by generating a very large Key-Value (KV) cache. This dynamic memory footprint adds significantly to the overall memory pressure, intensifying the swapping problem and increasing the computational load on the already struggling CPU.

7.2 Containerization and Network Overhead

Docker's default bridge networking, which utilizes Network Address Translation (NAT) for port mapping, introduces a minor latency overhead. This overhead is typically in the order of milliseconds and is not a primary contributor to a one-minute delay. While it can accumulate across multiple inter-container calls, its impact is negligible compared to the severe LLM inference bottleneck.

7.3 ChromaDB Performance Considerations

ChromaDB is employed as the vector database, a critical component for the retrieval aspect of the RAG system. Its reported memory usage is extremely low, at only 7 MB [Image 3], indicating that it is not a direct cause of the current memory pressure. For small to moderate datasets, ChromaDB is designed for fast and efficient vector search.

However, ChromaDB's performance relies on its embedding index residing in RAM.[4] If the embedding collection were to grow significantly (e.g., to millions of vectors) and compete for memory on a resource-constrained system, it could begin swapping to disk and subsequently become a performance bottleneck.

7.4 RAG System Efficiency

The overall efficiency of a RAG system is dependent on both the accuracy of its retrieval mechanisms and the quality of the context provided to the LLM.[9] While the primary latency is attributed to LLM inference, an inefficient retrieval process could indirectly contribute to performance degradation. This might occur by providing irrelevant or excessively long contexts to the LLM, which would increase the LLM's input token count, leading to a larger KV cache and a higher computational load, further stressing the already overloaded CPU.

RAG systems can also suffer from "topic gaps," where essential information is missing, or "capability failures," where the system cannot process information in specific ways.[1] These issues directly impact the

8. Risks and Limitations

8.1 Technical Limitations

- The current mobile-grade Intel i7-1165G7 CPU is not architecturally optimized for the highly parallel computations and high memory bandwidth required by a 7.6 billion parameter LLM.
- The critically insufficient 7.82 GB of *available* RAM for the 8.8 GB Ollama process forces the system into extensive memory swapping to disk, which is orders of magnitude slower than RAM access.
- The DeepSeek model's 131,072 token context length generates a very large Key-Value (KV) cache, significantly increasing memory pressure and intensifying the swapping problem.

8.2 Operational Constraints

- The current setup on a single laptop acting as a server is suitable for prototyping but inadequate for production-level concurrency, reliability, and sustained performance.
- Achieving acceptable LLM inference speeds (e.g., sub-10-second responses) necessitates substantial hardware upgrades, representing a significant investment.
- Transitioning to a production environment will require implementing high availability, robust monitoring, and comprehensive logging, which adds complexity.

8.3 Mitigation Strategies

- **Hardware Enhancement Strategies:**
  - **Immediate Priority:** Increase available physical RAM (e.g., to 64GB or more) to ensure ample headroom and prevent memory thrashing to disk.
  - **Long-Term Solution:** Integrate a dedicated NVIDIA GPU (e.g., RTX 4090, RTX 3090, or professional A40/A100) with at least 24GB VRAM into the server system for dramatic speedup in LLM inference.
  - Consider upgrading to a desktop-grade CPU with more cores (e.g., Intel Core i9 or AMD Ryzen 9) if a dedicated GPU is not immediately feasible.
  - Upgrade to a high-speed NVMe SSD for overall system responsiveness, especially for disk-intensive operations.
- **Software and Configuration Tuning:**
  - **Ollama and Model-Specific Optimizations:** If the application's use case permits, significantly reduce the DeepSeek model's context length (e.g., to 4096 or 8192 tokens) within the Ollama configuration.[1] Ensure Ollama is configured to utilize a dedicated GPU effectively (if added). Regularly update the Ollama installation.
  - **Docker Networking and Resource Allocation Adjustments:** For the isolated on-prem environment, consider running the ollama-llm and self-hosted-chatbot-backend containers with --net=host to bypass Docker's NAT overhead. Maximize the allocated CPU

cores and RAM for the Docker VM if Docker Desktop is used on Windows.
    - **ChromaDB Performance Tuning:** Monitor its memory footprint to ensure the embedding index remains in RAM, particularly as the knowledge base grows.
- **RAG Pipeline Refinements:**
    - **Retrieval Optimization:** Optimize how documents are split into chunks (smart chunking and overlap). Implement a re-ranking step after initial vector similarity search to improve relevance.
    - **Prompt Engineering:** Develop clear, concise, and effective prompts for the LLM.
    - **Systematic Evaluation Framework:** Establish a robust testing and evaluation framework (e.g., utilizing Ragas) to measure metrics like factual accuracy, answer relevance, and context utilization.
    - **Address Topic and Capability Gaps:** Analyze user query patterns and system failures to identify if the RAG system is lacking information ("Topic Gaps") or the ability to process information in specific ways ("Capability Failures").
- **Scalability and Robustness for Production Deployment:**
    - Transition to dedicated server infrastructure utilizing enterprise-grade components.
    - Implement high availability and redundancy strategies (e.g., running multiple Ollama instances, distributed ChromaDB).
    - Implement comprehensive monitoring tools and proactive alerting mechanisms.
    - Implement robust, centralized logging for all components.
    - Enforce strict security best practices.
    - Establish a clear plan for regular maintenance and updates.

9. Conclusion

The analysis indicates that the RAG-based self-hosted chatbot, while conceptually sound and containerized for portability, is currently operating as a prototype far from production readiness. The observed response latency exceeding one minute is primarily attributable to a critical mismatch between the demanding DeepSeek R1 7B Large Language Model (especially with its large 131,072 token context window) and the under-provisioned mobile CPU, compounded by severely insufficient available physical RAM. This resource deficit leads to extensive memory thrashing, where the system constantly swaps data between slow disk storage and RAM, fundamentally crippling performance.

The testing conducted in an isolated LAN environment using a TENDA Wi-Fi setup, where two client systems connected to a single server system, further highlighted this performance bottleneck, consistently showing response times exceeding one minute.

To achieve acceptable performance and ensure the feasibility of an on-premise deployment, the following foundational steps are critically required:

1. **Immediate Increase in Available Physical RAM:** This is paramount to eliminate memory thrashing, which is the most significant contributor to the current latency.
2. **Integration of Dedicated GPU Acceleration:** For truly responsive LLM inference, a dedicated GPU with ample VRAM is indispensable. This will provide the necessary parallel processing power and high-bandwidth memory that a CPU-only setup cannot deliver.

Following these essential hardware upgrades, strategic software tuning, such as reducing the LLM's context window and optimizing Docker networking, will yield further performance gains. Concurrently, refining the RAG pipeline through smart chunking, re-ranking, and systematic evaluation will enhance the quality and relevance of the chatbot's responses. Finally, transforming the prototype into an enterprise-grade solution necessitates a significant investment in dedicated server infrastructure, high availability measures, robust monitoring, comprehensive logging, and adherence to stringent security best practices. Without these foundational changes, the system will remain functionally limited, preventing it from meeting the performance and reliability demands of a business unit's on-premise requirements.